

Luồng 1: User — Authentication & Profile

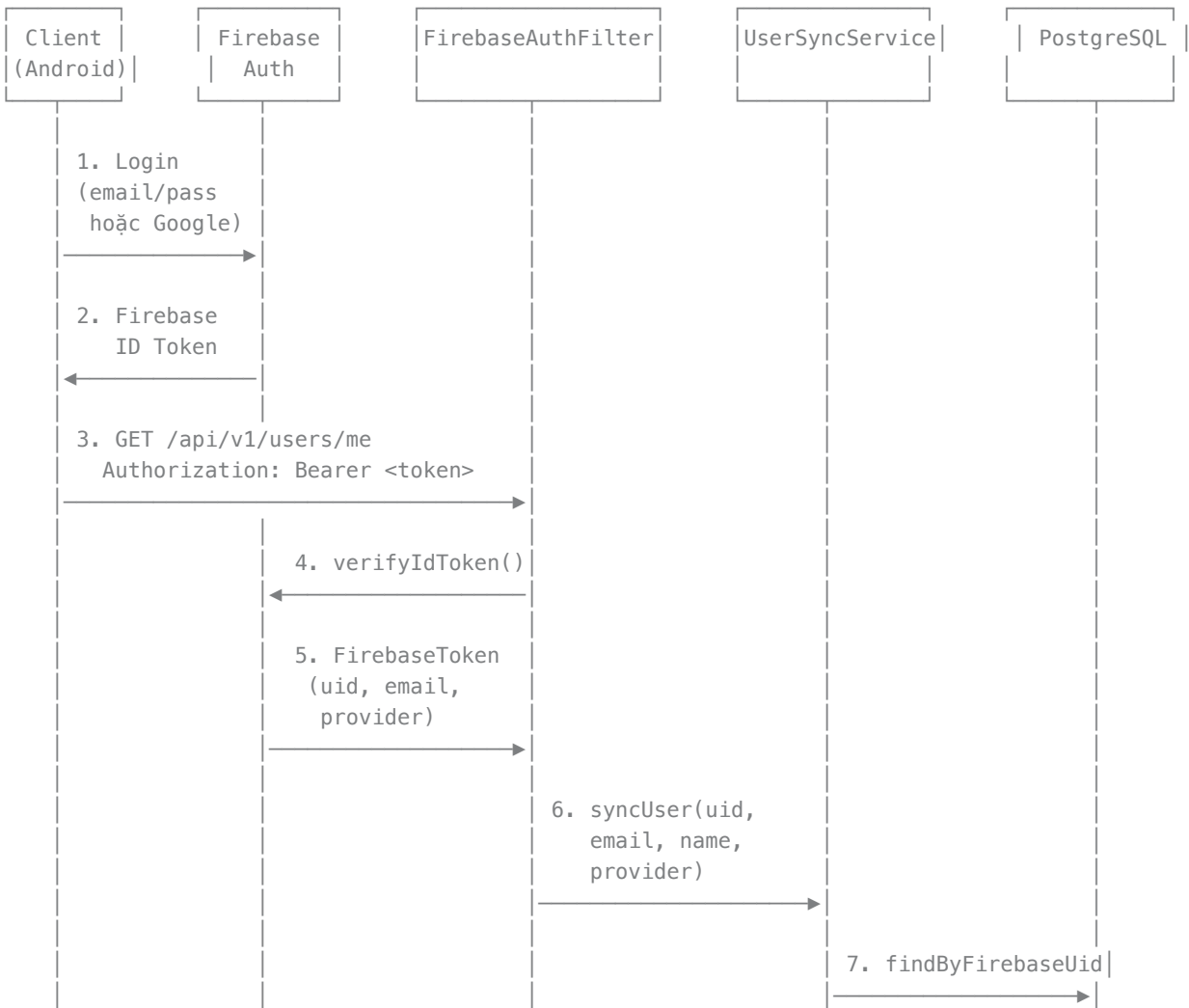
Mô tả chi tiết luồng xác thực, đăng ký, đồng bộ user và quản lý profile.

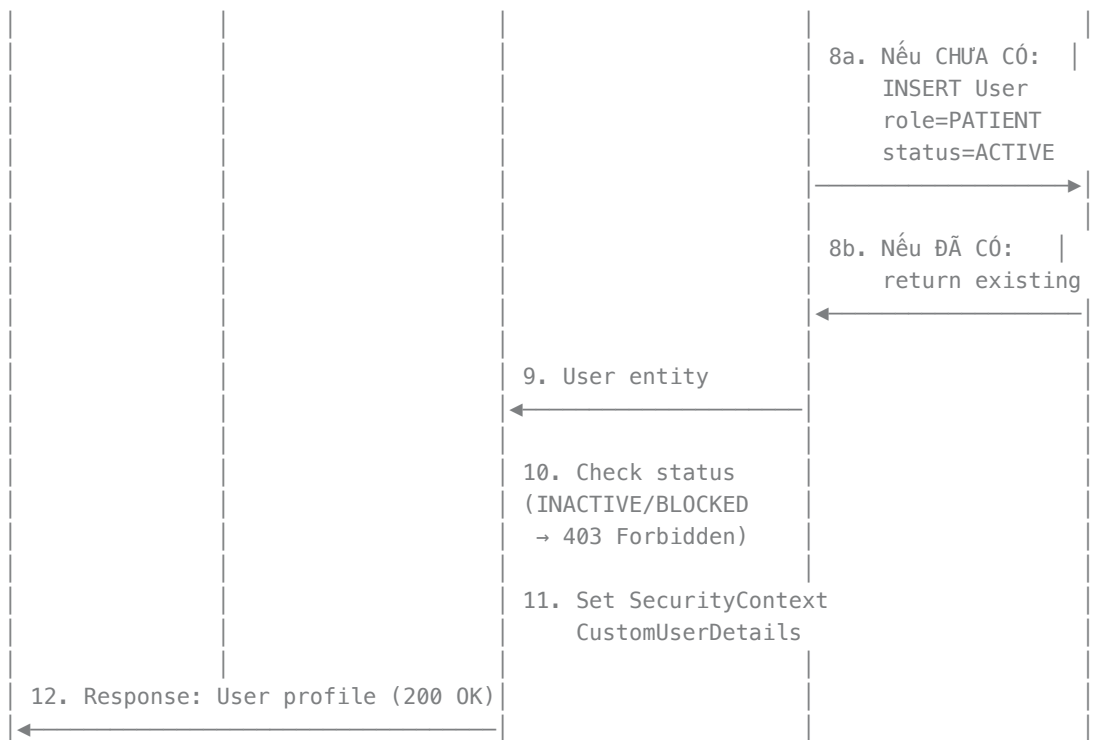
1.1 Tổng quan

Hệ thống sử dụng **Firebase Authentication** làm identity provider. Mọi request (trừ `/api/v1/public/**`) đều yêu cầu Firebase ID Token trong header `Authorization: Bearer <token>` .

Khi user login lần đầu, hệ thống **tự động tạo** record trong PostgreSQL với role `PATIENT` .

1.2 Sequence Diagram — Đăng nhập & Auto-sync





1.3 Chi tiết từng Step trong Code

Step 1-2: Client login với Firebase

- Client (Android) gọi Firebase SDK để đăng nhập (email/password hoặc Google Sign-In)
- Firebase trả về **ID Token** (JWT) chứa uid , email , name , sign_in_provider

Step 3: Client gửi request với Bearer Token

- Mọi request đều gửi header: Authorization: Bearer <firebase-id-token>

Step 4-5: FirebaseAuthFilter verify token

File: security/FirebaseAuthFilter.java

```
// Extract Bearer token từ header
String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);
String idToken = authHeader.substring("Bearer ".length());

// Verify với Firebase Admin SDK
FirebaseToken decodedToken = FirebaseAuth.getInstance().verifyIdToken(idToken);

// Extract thông tin user
String uid = decodedToken.getUid();
String email = decodedToken.getEmail();
String name = decodedToken.getName();
String provider = extractSignInProvider(decodedToken); // "google.com", "password", etc.
```

Xử lý lỗi:

- Token expired → HTTP 401 {"error": "Token expired"}
- Token invalid → HTTP 401 {"error": "Invalid token"}
- Không có header → skip filter, request tiếp tục (sẽ bị Spring Security chặn nếu cần auth)

Step 6-9: UserSyncService auto-sync

File: service/UserSyncService.java

```
public User syncUser(String firebaseUid, String email, String displayName, String authProvider) {
    // Tìm user existing bằng Firebase UID
    Optional<User> existing = userRepository.findByFirebaseUid(firebaseUid);
    if (existing.isPresent()) {
        return existing.get(); // Đã có → return luôn
    }

    // Chưa có → auto-create với role PATIENT
    User newUser = User.builder()
        .firebaseUid(firebaseUid)
        .email(email)
        .role(Role.PATIENT)
        .status(AccountStatus.ACTIVE)
        .authProvider(authProvider)
        .build();
    return userRepository.save(newUser);
}
```

Logic quan trọng:

- Mỗi request đều gọi syncUser() → đảm bảo user luôn tồn tại trong PG
- User mới tự động có role=PATIENT, status=ACTIVE
- Admin/Doctor KHÔNG được auto-create, phải tạo thủ công

Step 10: Check account status

File: security/FirebaseAuthFilter.java

```
if (user.getStatus() == AccountStatus.INACTIVE || user.getStatus() == AccountStatus.BLOCKED) {
    response.setStatus(HttpServletResponse.SC_FORBIDDEN);
    response.getWriter().write("{\"error\": \"Account is disabled\"}");
    return; // Không cho tiếp tục
}
```

Step 11: Set SecurityContext

File: security/FirebaseAuthFilter.java + security/CustomUserDetails.java

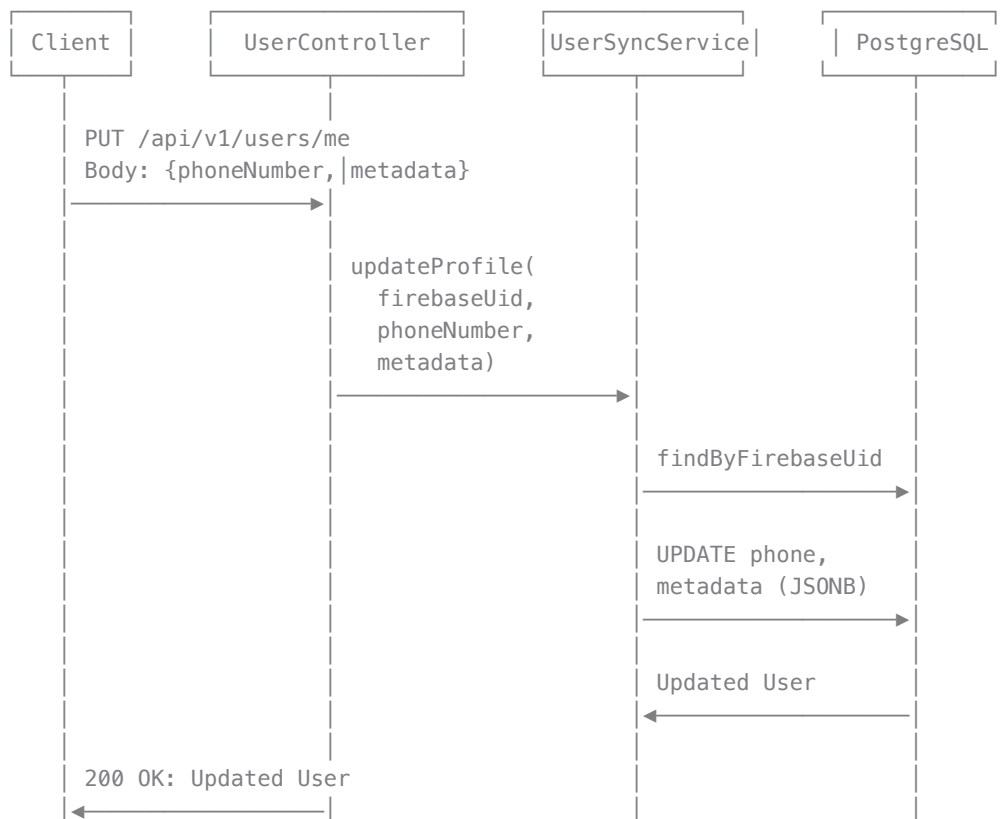
```
// Build CustomUserDetails chứa id, firebaseUid, email, role
CustomUserDetails userDetails = CustomUserDetails.build(user);
```

```
// Set vào Spring SecurityContext
UsernamePasswordAuthenticationToken authentication =
    new UsernamePasswordAuthenticationToken(userDetails, decodedToken, userDetails.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(authentication);
```

CustomUserDetails implements UserDetails , cung cấp:

- getAuthorities() → ROLE_PATIENT , ROLE_DOCTOR , hoặc ROLE_ADMIN
- getFirebaseUid() → dùng để query user-specific data
- getId() → UUID trong PostgreSQL

1.4 Sequence Diagram — Cập nhật Profile



Chi tiết code

File: service/UserSyncService.java

```
public User updateProfile(String firebaseUid, String phoneNumber, UserMetadata metadata) {
    User user = userRepository.findByFirebaseUid(firebaseUid)
        .orElseThrow(() -> new IllegalArgumentException("User not found"));

    if (phoneNumber != null) user.setPhoneNumber(phoneNumber);
    if (metadata != null) user.setMetadata(metadata);
```

```
    return userRepository.save(user);  
}
```

UserMetadata là JSONB object chứa thông tin bổ sung tùy theo role:

- PatientMetadataDto : fullName, dateOfBirth, gender, address, bloodType, allergies, medicalHistory
- DoctorMetadataDto : fullName, specialization, hospital, licenseNumber, yearsOfExperience, bio
- AdminMetadataDto : fullName, department

1.5 Các API trong luồng User

Method	Path	Mô tả
GET	/api/v1/users/me	Lấy profile user hiện tại
PUT	/api/v1/users/me	Cập nhật profile (phone, metadata)
POST	/api/v1/users/register-firebase	Đăng ký thông tin bổ sung
PUT	/api/v1/users/doctor/profile	Doctor cập nhật profile chuyên môn

File: 01_flow_user.md | Project: CareTalk Backend